

Unit 1: Basic Data Structures — Detailed Notes

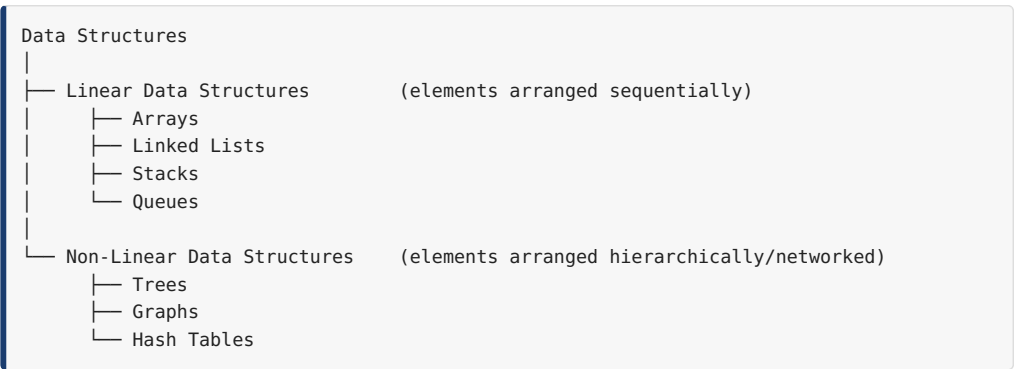
1. Introduction to Data Structures

A **Data Structure** is a way of organizing, storing, and managing data so that it can be accessed and modified efficiently.

Why Data Structures Matter

- Efficient data storage and retrieval
- Efficient use of memory
- Faster processing of data
- Foundation for algorithm design (searching, sorting, graph traversal, etc.)

Classification of Data Structures



Linear vs Non-Linear

Feature	Linear	Non-Linear
Arrangement	Sequential	Hierarchical/Networked
Traversal	Single pass	Multiple paths possible
Memory	Easy to manage	Complex relationships
Examples	Array, Stack, Queue	Tree, Graph

2. Arrays

An **array** is a collection of elements of the **same data type** stored in **contiguous memory locations**, accessed using an **index**.

2.1 One-Dimensional (1D) Arrays

Declaration (C):

```
int arr[5] = {10, 20, 30, 40, 50};
```

Declaration (Python — using list as array):

```
arr = [10, 20, 30, 40, 50]
print(arr[2]) # Output: 30 (index starts at 0)
```

Memory Representation:

Index:	0	1	2	3	4
Value:	10	20	30	40	50
Addr:	100	104	108	112	116

(assuming int = 4 bytes)

Address Calculation Formula:

$$\text{Address}(\text{arr}[i]) = \text{Base_Address} + i * \text{Size_of_element}$$

Basic Operations: | Operation | Description | Time Complexity | |—|—|—| | Access | Get element at index i | $O(1)$ | | Search | Find an element | $O(n)$ | | Insertion | Insert at end / at position | $O(1)$ / $O(n)$ | | Deletion | Delete at end / at position | $O(1)$ / $O(n)$ | | Traversal | Visit each element | $O(n)$ |

Example: Insert element at a given position

```
def insert(arr, pos, value):
    arr.insert(pos, value) # shifts elements right
    return arr

arr = [1, 2, 4, 5]
print(insert(arr, 2, 3)) # [1, 2, 3, 4, 5]
```

2.2 Multi-Dimensional Arrays

A **2D array** (matrix) is an array of arrays — useful for tables, grids, images, matrices.

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
print(matrix[1][2]) # Output: 6 (row 1, column 2)
```

Address Calculation for 2D array (Row-Major Order):

$$\text{Address}(\text{arr}[i][j]) = \text{Base} + (i * \text{NumCols} + j) * \text{Size_of_element}$$

Example: Matrix Addition

```
A = [[1, 2], [3, 4]]
B = [[5, 6], [7, 8]]
C = [[A[i][j] + B[i][j] for j in range(2)] for i in range(2)]
print(C) # [[6, 8], [10, 12]]
```

3. String Operations & String Algorithms

A **string** is a sequence of characters, often treated as an array of characters (`char[]` in C) or a built-in object (Python `str`).

3.1 Basic String Operations

```
s = "hello world"

len(s)          # Length -> 11
s.upper()       # "HELLO WORLD"
s[::-1]         # Reverse -> "dlrow olleh"
s.split()       # ['hello', 'world']
s.replace("hello", "hi") # "hi world"
s + "!"         # Concatenation -> "hello world!"
```

3.2 Common String Algorithms

a) Check if a String is a Palindrome

```
def is_palindrome(s):
    return s == s[::-1]

print(is_palindrome("madam")) # True
```

b) Reverse a String (manual, without slicing)

```
def reverse_string(s):
    result = ""
    for ch in s:
        result = ch + result
    return result
```

c) Count Character Frequency

```
def char_frequency(s):
    freq = {}
    for ch in s:
        freq[ch] = freq.get(ch, 0) + 1
    return freq

print(char_frequency("banana"))
# {'b': 1, 'a': 3, 'n': 2}
```

d) Pattern Matching — Naive String Search

```
def naive_search(text, pattern):
    n, m = len(text), len(pattern)
    positions = []
    for i in range(n - m + 1):
        if text[i:i+m] == pattern:
            positions.append(i)
    return positions

print(naive_search("abcabcabc", "cab")) # [2, 5]
```

Time Complexity: $O(n*m)$ — worst case (better algorithms: KMP $O(n+m)$, Rabin-Karp)

4. Linked Lists

A **Linked List** is a linear data structure where elements (**nodes**) are stored non-contiguously, and each node contains **data** and a **pointer/reference** to the next node.

4.1 Singly Linked List (SLL)

Each node points only to the **next** node.

```
[Data|Next] -> [Data|Next] -> [Data|Next] -> NULL
```

Implementation (Python):

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node

    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")

# Usage
sll = SinglyLinkedList()
sll.insert_at_end(10)
sll.insert_at_end(20)
sll.insert_at_end(30)
sll.display()    # 10 -> 20 -> 30 -> None
```

4.2 Doubly Linked List (DLL)

Each node has **two** pointers: **prev** and **next**, allowing traversal in both directions.

```
NULL <- [Prev|Data|Next] <-> [Prev|Data|Next] <-> [Prev|Data|Next] -> NULL
```

```
class DNode:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
```

```

def __init__(self):
    self.head = None

def insert_at_end(self, data):
    new_node = DNode(data)
    if not self.head:
        self.head = new_node
        return
    temp = self.head
    while temp.next:
        temp = temp.next
    temp.next = new_node
    new_node.prev = temp

def display_forward(self):
    temp = self.head
    while temp:
        print(temp.data, end=" <-> ")
        temp = temp.next
    print("None")

```

4.3 Circular Linked List (CLL)

The **last node** points back to the **first node** instead of `NULL` — forms a circle. Can be singly or doubly circular.

```

[Data|Next] -> [Data|Next] -> [Data|Next] -+
      ^_____|

```

```

class CircularLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            new_node.next = self.head
            return
        temp = self.head
        while temp.next != self.head:
            temp = temp.next
        temp.next = new_node
        new_node.next = self.head

    def display(self, count_limit=10):
        temp = self.head
        c = 0
        while temp and c < count_limit:
            print(temp.data, end=" -> ")
            temp = temp.next
            c += 1
        print("(back to head)")

```

4.4 Array vs Linked List

Feature	Array	Linked List
Memory	Contiguous	Non-contiguous (scattered)
Size	Fixed (static)	Dynamic

Access	O(1) random access	O(n) sequential access
Insertion/Deletion	Costly (shifting)	Easy (pointer change)
Extra Memory	None	Needs pointer storage

5. Stacks

A **Stack** is a linear data structure that follows **LIFO** (Last In, First Out).

Core Operations: `push()`, `pop()`, `peek()/top()`, `isEmpty()`

```

Push -> | 30 | <- Top
        | 20 |
        | 10 |
        -----

```

5.1 Stack using Array

```

class StackArray:
    def __init__(self):
        self.stack = []

    def push(self, item):
        self.stack.append(item)

    def pop(self):
        if self.is_empty():
            return "Stack Underflow"
        return self.stack.pop()

    def peek(self):
        if self.is_empty():
            return None
        return self.stack[-1]

    def is_empty(self):
        return len(self.stack) == 0

s = StackArray()
s.push(10); s.push(20); s.push(30)
print(s.pop())    # 30
print(s.peek())   # 20

```

5.2 Stack using Linked List

```

class StackLinkedList:
    def __init__(self):
        self.top = None

    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node

    def pop(self):
        if not self.top:

```

```

        return "Stack Underflow"
    popped = self.top.data
    self.top = self.top.next
    return popped

```

5.3 Applications of Stack

- Function call management (call stack, recursion)
- Undo/Redo operations in editors
- Expression evaluation (Infix → Postfix → Prefix)
- Balanced parentheses checking
- Backtracking algorithms (maze solving, DFS)
- Browser history (back button)

Example: Balanced Parentheses Check

```

def is_balanced(expr):
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}'}
    for ch in expr:
        if ch in "([{":
            stack.append(ch)
        elif ch in ")]}":
            if not stack or stack.pop() != pairs[ch]:
                return False
    return len(stack) == 0

print(is_balanced("[]{()}")) # True
print(is_balanced("[{([)]}")) # False

```

6. Queues

A **Queue** is a linear data structure that follows **FIFO** (First In, First Out).

Core Operations: `enqueue()`, `dequeue()`, `front()`, `isEmpty()`

Front -> | 10 | 20 | 30 | <- Rear
 Dequeue from Front, Enqueue at Rear

6.1 Queue using Array

```

class QueueArray:
    def __init__(self):
        self.queue = []

    def enqueue(self, item):
        self.queue.append(item)

    def dequeue(self):
        if self.is_empty():
            return "Queue Underflow"
        return self.queue.pop(0)

    def is_empty(self):
        return len(self.queue) == 0

```

```
q = QueueArray()
q.enqueue(10); q.enqueue(20); q.enqueue(30)
print(q.dequeue()) # 10
```

6.2 Queue using Linked List

```
class QueueLinkedList:
    def __init__(self):
        self.front = None
        self.rear = None

    def enqueue(self, data):
        new_node = Node(data)
        if not self.rear:
            self.front = self.rear = new_node
            return
        self.rear.next = new_node
        self.rear = new_node

    def dequeue(self):
        if not self.front:
            return "Queue Underflow"
        temp = self.front
        self.front = self.front.next
        if not self.front:
            self.rear = None
        return temp.data
```

6.3 Types of Queues

Type	Description
Simple Queue	Basic FIFO
Circular Queue	Rear connects back to front, reuses empty space
Priority Queue	Elements served based on priority, not order
Double-Ended Queue (Deque)	Insertion/deletion from both ends

6.4 Applications of Queue

- CPU/Process scheduling (Round Robin)
- Handling requests in web servers / printers (print queue)
- Breadth-First Search (BFS) in graphs/trees
- Buffering (IO Buffers, streaming data)
- Call center systems (customer call handling)

7. Time Complexity

Time Complexity measures how the **execution time** of an algorithm grows as the input size (n) increases. It is *not* actual runtime — it's a way to compare algorithm efficiency independent of hardware.

Common Time Complexities (Best to Worst)

Notation	Name	Example
----------	------	---------

O(1)	Constant	Array access
O(log n)	Logarithmic	Binary Search
O(n)	Linear	Linear Search
O(n log n)	Linearithmic	Merge Sort, Quick Sort
O(n ²)	Quadratic	Bubble Sort, Insertion Sort
O(2 ⁿ)	Exponential	Recursive Fibonacci
O(n!)	Factorial	Traveling Salesman (brute force)

Example:

```
# O(1) - Constant Time
def get_first(arr):
    return arr[0]

# O(n) - Linear Time
def linear_search(arr, target):
    for i, val in enumerate(arr):
        if val == target:
            return i
    return -1

# O(n^2) - Quadratic Time
def print_pairs(arr):
    for i in arr:
        for j in arr:
            print(i, j)
```

8. Space Complexity

Space Complexity measures the total amount of **memory** an algorithm needs relative to input size, including: - Input space (space taken by inputs) - Auxiliary space (extra/temporary space used during execution)

Space Complexity = Input Space + Auxiliary Space

Example:

```
# O(1) space - no extra memory used, regardless of input size
def sum_array(arr):
    total = 0
    for num in arr:
        total += num
    return total

# O(n) space - new array of size n is created
def double_values(arr):
    return [x * 2 for x in arr]
```

Recursive functions often use **O(n) space** due to the **call stack**:

```
def factorial(n):
    if n == 0:
        return 1
```

```
return n * factorial(n - 1) # O(n) space on call stack
```

9. Big-O Notation

Big-O Notation describes the **upper bound** (worst-case growth rate) of an algorithm's time or space requirement as input size grows towards infinity.

Why Big-O?

- Allows comparing algorithms independent of hardware/language
- Focuses on **growth rate**, ignoring constants and lower-order terms

Rules for Calculating Big-O: 1. Drop constants: $O(2n) \rightarrow O(n)$ 2. Drop lower-order terms: $O(n^2 + n) \rightarrow O(n^2)$ 3. Different inputs get different variables: $O(a + b)$ not $O(2n)$

Example:

```
def example(arr):
    print(arr[0])          # O(1)
    for x in arr:          # O(n)
        print(x)
    for x in arr:          # O(n)
        for y in arr:      # O(n)
            print(x, y)
# Total: O(1) + O(n) + O(n^2) = O(n^2) (dominant term wins)
```

Big-O vs Big-Ω vs Big-Θ

Notation	Meaning	Case
Big-O (O)	Upper bound	Worst case
Big-Omega (Ω)	Lower bound	Best case
Big-Theta (Θ)	Tight bound	Average case

10. Hands-on Session Summary

Task	What You Practice
Implement Arrays & Linked Lists	Insertion, deletion, traversal, memory model
Stack & Queue using Array/Linked List	LIFO vs FIFO logic, dynamic vs static memory
Analyze Time & Space Complexity	Identify Big-O for loops, recursion, nested structures

Exercise Questions (20)

Section A: Conceptual 1. What is a data structure? Differentiate between linear and non-linear data structures with examples. 2. Explain the difference between an array and a linked list. When would you prefer one over the other? 3. What is the difference between a Singly Linked List, Doubly Linked List, and Circular Linked List? 4. Define Stack and Queue. State their core principle

(LIFO/FIFO) with a real-world example each. 5. What is Big-O notation, and why is it used instead of measuring actual execution time? 6. Differentiate between Time Complexity and Space Complexity with examples. 7. Explain row-major order for storing a 2D array and derive the address formula for an element `arr[i][j]`. 8. What are the applications of stacks and queues in real-world computing systems? 9. Explain the concept of a Circular Queue and why it is preferred over a Simple Queue in some situations. 10. What is the difference between Big-O, Big-Omega, and Big-Theta notations?

Section B: Problem Solving / Programming 11. Write a program to insert, delete, and traverse elements in a singly linked list. 12. Write a program to reverse a singly linked list without using extra space. 13. Implement a stack using an array and demonstrate push, pop, and peek operations. 14. Implement a queue using a linked list and demonstrate enqueue and dequeue operations. 15. Write a program to check whether a given string of brackets `(){}[]` is balanced, using a stack. 16. Write a program to check if a given string is a palindrome without using built-in reverse functions. 17. Given an array of `n` integers, write an algorithm to find the maximum and minimum elements, and state its time complexity. 18. Write a program to convert an infix expression to postfix expression using a stack. 19. Implement a circular queue using an array, handling the “queue full” and “queue empty” conditions correctly. 20. Analyze the following code and determine its time complexity:

```
def mystery(n):
    count = 0
    for i in range(n):
        for j in range(i, n):
            count += 1
    return count
```

End of Unit 1 Notes